1 Network mode

This section documents the multi-station network estimator (`ProcessingMode::PPP_RTK_Network`) that *produces* the satellite-side corrections the single-receiver user modes consume. Where every other mode in this reference *differences away* the satellite clock and hardware biases (the differencing section) or imports them as known SSR quantities (the SSR-corrections section), the network producer turns that relationship around: it estimates the satellite clock and the satellite *phase* and *code* biases *explicitly*, jointly across a network of stations whose coordinates are known, and emits them in IGS-SSR convention for a downstream consumer. The closed loop is deliberate: the same single-receiver PPP-RTK engine documented elsewhere becomes the product test, run on a station *held out* of the network solve.

Scope and status. The numerical core, the arrowhead normal-equation kernel (`filter/NetworkNormalEquations`) and the semantic estimator that drives it (`filter/NetworkEstimator`), is implemented and validated on a synthetic noise-free network (truth recovery up to the declared S-transform). The full pipeline is live: the `PPP_RTK_Network` enum, its forced-uncombined `DifferencingMode::None` (`measurement/MeasurementBuilder`), the explicit satellite phase-bias state in `filter/StateManager`, and the mode dispatch in `Application::run` all exist. `Application::runNetwork` is no longer the estimator: it is a thin epoch-loop orchestrator that calls a fixed ordered sequence of single-concern seams (Section 1.1), and every step this chapter derives, including per-station design-row assembly from each station's `MeasurementModel` (`networkPassAHead_` / `networkPassB_`), the S-basis datum with its null-space rank gate (Section 1.4, in `networkSolveGlue`), the between-epoch time update, and the product emitter (`NetworkProductEmitter`, driven from `networkEmitStore`) runs inside one of them. The mathematics below describes the implemented estimator core and the locked design.

This chapter treats the single-epoch solve. The between-epoch carry is layered, and the layers differ in status. The base *diagonal* D2 shared carry, each shared parameter's posterior value and variance re-injected as a diagonal prior on the next epoch's shared border, inflated by $q\,\Delta t$ and dropped on a datum change (`networkInjectD2CarryPrior`, `app/NetworkRunContext`), is *unconditional*: it carries no enable flag, so the shipped producer is a filtered, not a snapshot, solve. The upgrades that ride on top of it, namely the C1 full-covariance shared carry (`network.shared_carry_full_cov`, folded through `addSharedPriorDense`), the Card-4 invariant DD-carry (`network.carry_invariant`, `setDdPriors`), the station-local phase-leveling / ZWD / receiver-code-bias carries folded through `addStationPrior`, and the MW-WL / EWL / lane-coordinate widelane layers, are each large default-off subsystems in their own right and are only referenced here, not derived.

1.1 Producer architecture: the decomposed epoch loop

The rest of this chapter is mathematics. This subsection is the map that says where each piece of it executes, because the answer is no longer "in one function".

Why the shape changed. The producer began as a single driver function that reached roughly 14,700 lines. At that size it could not be reasoned about as a whole, could not be exercised below the granularity of an entire run, and could not be reviewed: a reader looking for the datum build had no way to bound where it was *not*. It was therefore decomposed into single-concern *seams*, and

the decomposition chose the strongest available safety property. Relocation is *verbatim*: bodies move, they are not rewritten, and each increment was gated on a matrix of golden runs required to be *byte-identical* to the pre-cut binary's output. That gate is what licenses the claim that the equations below still describe the code: the driver shrank by roughly a factor of eight, to under 1,800 lines of epoch-loop orchestration, without the numbers moving. It has since grown back to roughly 2,250 lines as later cards added seams to the call sequence, which is exactly the pressure the placement rules of this section exist to resist. The gate's coverage should be stated plainly, though: `docs/network_decomposition_plan.md` records per-increment byte gates through increment 7b only, and the later cuts (solve glue, carry install, DD-prior build, producer AR, MW/WL glue, re-level, teardown) landed with no recorded gate in that status section.

Four stages. A network run decomposes into four stages: three translation units and the seam sequence between them.

- **Setup** (`app/ApplicationNetworkSetup`). `networkValidateInputs_` rejects an unrunnable configuration before anything is allocated; `networkSetup_` then builds the station roster and its synchroniser, the product sink and its declared S-basis contract, and, when the closed loop is armed, the consumer engine (Section 1.7). Its output is one `NetworkSetupResult`, filled in place and deliberately neither copyable nor movable: three run-long references pin its member addresses, so the member declaration order in that header is normative rather than cosmetic.
- **Assembly** (`app/ApplicationNetworkAssembly`), two passes per epoch. PASS A is *layout*: it runs each present station's geometry models and pre-allocates the shared satellite states, taking the datum (pivot) station *first* so that the phase-bias columns admitted to the shared block are exactly those backed by a pivot-receiver ambiguity pin, since the local half of the datum (Section 1.4) constrains the column layout, not merely the solve. PASS B is *assembly*: each station's undifferenced design block is computed with no receiver-side differencing and then *split*, the satellite clock and phase-bias columns scattering into the shared border $\boldsymbol{A}_{s,r}$ and the remainder becoming the private block $\boldsymbol{A}_{r,r}$ of Equation (1); the three receiver-position columns are dropped, because network station coordinates are held fixed. Equation (4) is formed from exactly these two pieces.
- **The epoch seams**: the ordered sequence tabulated below, which *is* the body of the loop.
- **Teardown** (`filter/NetworkTeardown`). The run-end readout, the service counters and the summary writers live in `networkTeardown`, which also returns the process exit code; the driver's tail is literally a `return` of that call.

The seam contract. A seam is a free function over the run context, the configuration, the logger and the trace bundle, opening with an alias preamble that reconstructs the driver's former local names so the relocated body reads as it did before the cut. The contract each seam header states is that it *owns no state*: every carried object is a reference into the context. Two structural adaptations to pure relocation are worth recognising when reading the code. A relocated block that used to `continue` the epoch loop cannot, so it returns an `EpochAction` of `Proceed`, `SkipEpoch` or `StopRun` (`filter/NetworkRunTypes`), which keeps the per-skip bookkeeping verbatim inside the seam while the caller's `continue` preserves the original timing-bracket bypass. And a block whose locals were read further down the loop returns a result struct (`NetworkSolveGlueResult` and its siblings) that the call site re-aliases.

Why there is a context object. Carried state, configuration snapshots and counters live in one place: `NetworkRunContext` (`app/NetworkRunContext`), holding a `const NetworkKnobs` snapshot of the configuration, taken in the original getter read order, the cross-epoch `NetworkCarryState`, and `NetworkServiceCounters`. The rule is that a seam introduces no function-local carry: a relocated block that leaves a local behind silently *forks* the carry, and one decomposition pass was spent deleting the dead aliases that pattern produced. Persistence carries its own discipline, the *three-place rule*: a carry field that is a snapshot member must appear in the warm-resume fingerprint, in the restore, and in the snapshot write, all three in `NetworkRunContext`. A field present in only two of the three yields a warm resume that silently disagrees with the run that wrote it.

The ordered epoch seams. The table below lists the per-epoch call sequence in driver order. Each call is bracketed by the driver's section timers, and rows marked *default off* are inert unless explicitly armed, a point worth keeping in view, since most of the newer machinery in this producer is in that state.

seam call (driver order)	concern (translation unit)
`networkFastClockTick`	sub-epoch fast-clock tick between solution epochs; *default off* (`filter/NetworkFastClock`)
`networkPassAHead_`	PASS A head: per-station geometry and shared-state pre-allocation, datum station first (`app/ApplicationNetworkAssembly`)
`networkPassASlipScan_`	PASS A tail and the slip scans that fill the slip funnel (`app/ApplicationNetworkAssembly`, `slip/NetworkSlipQc`)
`networkAssemblyPrefilter_`	raw slip observables; cross-station observability prune (`app/ApplicationNetworkAssembly`)
`networkSignalIntegrity`	per-LOS band-state classifier: degraded/unvetted phase demotion and regain refloat; *default off* (`filter/NetworkSignalIntegrity`)
`networkMeasurementAdmission`	product-referenced corrupt-channel screen, the one admission test a differential screen cannot see; *default off* (`filter/NetworkMeasurementAdmission`)
`networkBuildPhaseDatumCandidates`, then `networkAnchorHygiene` and `networkElectPhaseAnchors` / `networkElectEnsembleDatum` / `networkElectChainDatum`	election of the *local* half of the datum: the candidate census and coverage gate, the eq34 reattach hygiene that governs anchor-host eligibility and the fresh-pin deweight ramp, then whichever realisation is armed (`filter/NetworkPhaseDatum`, `filter/NetworkAnchorHygiene`, `filter/NetworkChainDatum`)
`networkAssemblyRegisterColumns_`	clear the layout, register this epoch's shared columns, open the arrowhead system (`app/ApplicationNetworkAssembly`)
`networkSdEwlAr`	single-difference extra-widelane integer decision; *default off* (`filter/NetworkSdEwlAr`)
`networkRxCbDemeanPrepass`	receiver code-bias carry de-mean before assembly reads the map; *default off* (`app/NetworkRunContext`)
`networkPassB_`	PASS B produce/consume: the split design blocks handed to `addStation` (`app/ApplicationNetworkAssembly`, `filter/NetworkCoupling`)
`networkBuildEnsembleGauges`, `networkBuildCarryDdPriors`	sum-to-zero phase gauges; Card-4 DD carry priors (`filter/NetworkDdPriorBuild`)
`networkSolveGlue`	shared datum build, rank gate and drop-and-heal, D2 carry injection, the constrained solve, within-epoch RAIM re-solve, withhold gates (`filter/NetworkSolveGlue`, calling `NetworkSbasis`, `NetworkPhaseDatum`, `NetworkRelevel`)
`networkInstallCarry`	DD posterior read-back into the invariant carry; *default off* (`filter/NetworkInstallCarry`)
`networkProducerAr`	producer-side AR diagnostic and normal-equation export; read-only and log-only, *default off* (`filter/NetworkProducerAr`)
`networkSolPrep`	map to the solution epoch; per-satellite discontinuity signalling (`filter/NetworkSolPrep`)
`networkEmitStore`	emit re-base, phase-bias continuity filter, store push, product write, closed-loop consume (`filter/NetworkEmitStore`)
`networkFastClockHarvest`	fast-clock harvest off the accepted solution; *default off* (`filter/NetworkFastClock`)
`networkIonoDiag`, `networkDdFrac`, `networkShDevVtec`, `networkShHealth`, `networkIonoDwRelease`, `networkLeveling`	diagnostic sweeps, the iono-deweight release hygiene that reseeds slant states a zero-weight era left unvalidated, and the SH-deviation and phase-leveling carry harvests (`filter/NetworkIonoDiag`, `NetworkDdFrac`, `NetworkShDevVtec`, `NetworkShHealth`, `NetworkIonoDwRelease`, `NetworkLeveling`)
`networkPostFitRaim`	post-fit network RAIM and its attribution pass (`filter/NetworkRaim`)
`networkClockQc`	two-state clock update and clock fault QC (`filter/NetworkClockQc`)
`networkPostfitSigma0`, `networkShSpectrum`	post-fit variance factor and reduced-normal spectrum; *default off* (`filter/NetworkFrameDiag`)
`networkPrefitCensus`	the only always-on *pre*-solve screen: prefit innovation census, structural fuse, seed-birth tripwire and estimation-topology ledger (`filter/NetworkPrefitCensus`)
`networkRedundancyCensus`	row redundancy and testability, the statistical parked-garbage monitor; measure-only, *default off* (`filter/NetworkRedundancyCensus`)
`networkBlockBaardaEmit`	(station, satellite) LOS block outlier census; measure-only, *default off* (`filter/NetworkBlockBaarda`)
`networkCongruenceEmit`	station-congruence screen: coordinate-fault detection in position space; measure-only, *default off* (`filter/NetworkCongruence`)
`networkMdbFlush`	Baarda reliability numbers and minimal detectable bias on the

Reading key. The three derivations that follow land in three places in that sequence. The rank deficiency and its minimum-constraint datum (Section 1.4) are built and gated inside `networkSolveGlue`. The arrowhead Schur reduction (Section 1.5) happens in the kernel that the same seam invokes, on normals accumulated station-by-station during PASS B. The closed loop (Section 1.7), meaning emission, product write and, when armed, the held-out consumer step, is `networkEmitStore`. Everything else in the table is quality control, diagnostics or carry bookkeeping wrapped around those three.

1.2 Joint multi-station observation model

The producer runs on the same *undifferenced, uncombined* observations as single-receiver PPP (the observation-equations section), but stacks the equations of every station $r \in \mathcal{R}$ into one system. The defining structural fact is the partition of the unknowns into a block *shared* by all stations and a block *private* to each:

- **Shared (satellite-side).** The estimable satellite clock $c\,\mathrm{d}t^i$, the per-signal satellite code bias B^i_s, and the per-signal satellite *phase* bias $\bar{b}^i_s$. These columns are *identical* across every station, since a satellite's clock and hardware delays do not depend on who is observing it. (Orbits are held fixed to a precise product here; orbit determination is out of scope for the producer.) When `network.estimate_iono_sh` is set, the shared block carries a fourth family: the coefficients a_{nm}, b_{nm} of a global single-layer spherical-harmonic VTEC expansion (`SharedParamKind::VtecSh`, registered through `NetworkEstimator::registerVtecCoeff`), optionally a second, plasmasphere-side shell as well (`estimation.global_vtec_sh.layer2_max_degree`, default 0, i.e. single-layer; both shells' coefficients coexist on the shared border and the pin row sums them), network-wide rather than per-satellite. This co-estimated ionosphere model replaces the external GIM and forces satellite code-bias estimation on, since the two are jointly load-bearing for the per-satellite slant-iono columns below, and its columns Schur-reduce onto the shared border exactly like the satellite-indexed families.
- **Private (station-side).** For each station r: its receiver clock $c\,\mathrm{d}t_r$, the inter-system time offsets $c\,\delta t_r^{(g)}$, the zenith wet troposphere ZWD_r (and gradients), the per-satellite slant ionosphere $I^i_{r,1}$, the receiver biases, and one float ambiguity $N^i_{r,s}$ per tracked carrier, plus, only under the chain datum (`network.phase_datum_eq34`, default off), a per-(receiver, system, signal) receiver *phase*-bias split $\delta_{r,s}$ re-derived fresh each epoch (`StateKeys::makeReceiverPhaseBiasKey`), which is what stops the receiver's fractional phase bias lumping into the *emitted* satellite phase bias under that realisation. Station coordinates are *known* (the network constrains them), so position is not a free parameter.

Write the shared satellite-parameter vector as $\boldsymbol{x}_s$ and station r's private nuisance vector as $\boldsymbol{x}_r$. The undifferenced observation rows of station r linearise (the observation-equations section) to

$$\boldsymbol{y}_r \;=\; \boldsymbol{A}_{s,r}\,\boldsymbol{x}_s \;+\; \boldsymbol{A}_{r,r}\,\boldsymbol{x}_r \;+\; \boldsymbol{\varepsilon}_r, \qquad r \in \mathcal{R}, \tag{1}$$

where $\boldsymbol{y}_r$ is the prefit residual (observed minus computed), $\boldsymbol{A}_{s,r}$ is the Jacobian with respect to the *shared* satellite columns, and $\boldsymbol{A}_{r,r}$ is the Jacobian with respect to station r's *own* nuisances. Crucially there is no $\boldsymbol{A}_{r,r'}$ term for $r' \neq r$: a station's observations touch only the shared block and its own private block. That sparsity is the structural engine of Section 1.5.

No receiver-side differencing. Unlike the PPP-RTK user mode, the producer applies *no* between-satellite or between-receiver differencing. The differencing strategy is forced to `DifferencingMode:`

`:None` (`MeasurementBuilder.cpp`), overriding any `between_satellite_sd` toggle that may be present in a network config. Differencing would cancel exactly the satellite-side terms the producer is trying to *estimate*; the network instead carries them as explicit columns and resolves the resulting rank deficiency with a minimum-constraint datum (Section 1.4).

1.3 The explicit satellite phase-bias state

The single defining contrast with the user side is the treatment of the satellite phase bias b_s^i (here written $\bar{b}_s^i$ once promoted to a state). On the user side this term is never an explicit unknown: between-satellite single differencing (the differencing section) cancels the *receiver* phase bias, the satellite phase bias arrives as a *known* SSR product, and what remains is an integer single-difference ambiguity. The user handles biases *implicitly*.

The producer is what *generates* that known product, so it must carry the satellite phase bias as a genuine estimated state. The state vector therefore gains a per-(satellite, signal) phase bias column,

$$\bar{b}_s^i \in \boldsymbol{x}_s, \qquad \text{keyed by } \texttt{makeSatPhaseBiasKey}(i, s) = \texttt{"}i\texttt{|}s\texttt{|PB"}, \tag{2}$$

held in `StateManager::sat_phase_bias_indices_`, a map kept *deliberately disjoint* from the satellite *code* bias map (`sat_bias_indices_`); the trailing `|PB` discriminator keeps the two namespaces from colliding (`include/filter/StateKeySchema.hpp`). Its process noise is a tiny random walk (`network.process_noise.sat_phase_bias`, legacy `filter.process_noise.sat_phase_bias_q`; default two orders of magnitude below the code-bias density), reflecting that satellite phase hardware delays are nearly constant but not strictly so: a $q = 0$ random walk would over-converge the diagonal of $\boldsymbol{P}$ and leave the column deaf to genuine slow drift (`src/core/filter/StateManager.cpp`).

Remark 1. The estimated $\bar{b}_s^i$ is not the "raw" satellite phase hardware delay; that quantity is not separately estimable. It is the delay *lumped with the network pivot-receiver ambiguity/clock datum* (Odijk's δ^s, [1]): the estimable combination that, when removed by a user receiver, restores the integer property of that user's single-difference ambiguities. This lumping is exactly why a *consistent* phase-bias product matters (the SSR-corrections section): the producer's datum and the consumer's integer recovery must be the same S-system.

1.4 Rank deficiency and the minimum-constraint S-basis

The undifferenced network system Equation (1) is rank deficient. The same linear dependencies that the user side disposes of by differencing are present here as exact *gauge freedoms* of the joint normal matrix: there is no observation that distinguishes a common shift of all satellite clocks from an opposite shift of all receiver clocks, nor the split of an absolute phase bias between the satellite column and the per-station ambiguity datum, nor an inter-system bias (ISB) common to a constellation. These are precisely the S-system ambiguities of [2]: the data fix only *estimable combinations*, and a *datum* must be chosen to pin the remaining null space.

Exact gauge, soft elimination. The chosen resolution is a *minimum-constraint* datum applied as exact hard pins to the reduced shared system, never a hard structural elimination of columns. Concretely, each gauge is realised as a single *Dirichlet pin* of one shared column c to a datum value (almost always 0): the column's known contribution is moved to the right-hand side, its row and column in the reduced normal matrix are zeroed, and a diagonal entry is set so that $x_c =$ value holds exactly (`NetworkEstimator::solve`, `src/core/filter/NetworkEstimator.cpp`). The pin is applied to a *copy* of the reduced normals (the kernel's `solveSharedConstrained` seam), leaving

the estimator's own reduced system pristine so per-station back-substitution (Section 1.5) stays valid. The pinned diagonal is set commensurate with the reduced normals' own scale rather than to unity, so that a genuinely *under*-specified datum still surfaces as a near-zero *relative* pivot in the positive-definiteness guard, independent of the network's overall weighting magnitude.

In S-transform language, if $\boldsymbol{x}$ collects the rank-deficient undifferenced parameters and $\boldsymbol{S}$ is the S-basis selection, the emitted satellite corrections are the estimable combination $\boldsymbol{S}\,\boldsymbol{x}$; choosing the datum *is* choosing $\boldsymbol{S}$, and the hard pins above are the minimal set of rows of $\boldsymbol{S}$ that remove the deficiency. The two halves of the datum live in two places:

- **Shared-block gauges**, the overall clock datum (one per constellation), the per-constellation phase-bias↔ambiguity datum, and the ISB datum, are the elements that touch the shared satellite columns. These go through the explicit `NetworkEstimator::Datum` (a list of column pins) and are applied in `solve`.
- **Local datum elements**, a network *pivot receiver*'s own ambiguity/clock pin, which the emitted satellite phase bias absorbs (Odijk Eq. 4/9), live on that one station's private block. They are realised by the *caller* as tight prior rows folded into that station's $\boldsymbol{A}_{r,r}$, $\boldsymbol{W}$, and $\boldsymbol{y}$, not through the shared `Datum` (`NetworkEstimator.hpp`). Station priors are also what make each local block $\boldsymbol{N}_{rr}$ full rank for the Schur step (Section 1.5).

Selectable datum realisations. The pins just described are the *legacy default*; each half of the datum now admits alternatives, selected by config and recorded in the `SbasisDeclaration`, all producing the same estimable $\boldsymbol{S}\,\boldsymbol{x}$ but distributing the gauge differently. The clock/ISB datum offers three modes (`SbasisDeclaration::IsbDatum`): the hard `PerConstellationSatClockPin` above (default), a `SumToZeroPerConstellation` constraint (`network.clock_datum_sum_to_zero`) that spreads the pin across all satellites of a constellation, and a `BroadcastAnchoredPerSatellite` soft anchor (`network.clock_datum_broadcast_anchored`) tying each satellite clock to its broadcast value at a declared 1σ. The ambiguity↔phase-bias datum likewise offers three (`AmbDatum`): the local `PivotReceiverZeroAmbiguity` pin above (default), whose anchor may optionally be elected *per* (satellite, signal) at the highest-coverage tracking receiver rather than at the single clock pivot (`network.phase_datum_prioritized`, default off; `networkElectPhaseAnchors`, `filter/NetworkPhaseDatum`), which is the same one-pin-per-freedom constraint realised at a different station, so that a satellite low or unseen at the clock pivot still keeps a clean gauge; a `SumToZeroPerReceiver` ensemble (`network.phase_datum_ensemble`), and a `SingleReceiverChain` realisation (`network.phase_datum_eq34`). When satellite code biases are estimated (the W6 slice), their own gauge, a per-constellation sum-to-zero (`CodeBiasDatum::SumToZeroPerConstellation`), is folded as explicit constraint rows through `NetworkEstimator::addSharedConstraintRows`, not as column pins. Whichever realisations are active, the rank gate below is declared against the matching S-basis dimension.

Remark 2. A rigorous null-space rank gate, which eigen-decomposes the pre-constraint reduced normals and asserts that the count of small eigenvalues equals the *declared* S-basis dimension, is implemented as `NetworkSbasis::assertNullSpaceRank` and is invoked on `reducedSharedNormals()` *before* the constrained solve, against the `SbasisDeclaration`'s `declared_dimension` (the solve seam `networkSolveGlue`, `filter/NetworkSolveGlue`, which also builds the datum via `NetworkSbasis::buildSharedDatum`; see Section 1.1). It complements the positive-definiteness guard: the guard catches the exact-deficiency (under-specified datum) case on the pinned pivot, while the rank gate certifies the reduced deficiency against the declared gauge dimension and withholds any epoch whose null-space count disagrees. Under the prioritized phase datum the gate first attempts a *drop-and-heal*

pass, soft-anchoring exactly the near-null phase-bias columns (`healNearNullPhaseBias`, `filter/NetworkPhaseDatum`), and re-gates once; on the legacy pivot datum there is nothing to heal, the heal returns zero, and the epoch is withheld immediately.

1.5 Per-station Schur reduction (arrowhead normals)

Stacking Equation (1) over all stations and forming weighted normal equations $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{A}\,\boldsymbol{x} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{y}$ yields a *bordered block-arrow* ("arrowhead") system, because the station–station off-diagonal blocks are exactly zero (stations couple only through $\boldsymbol{x}_s$):

$$\begin{bmatrix} \boldsymbol{N}_{ss} & \boldsymbol{N}_{s1} & \boldsymbol{N}_{s2} & \cdots & \boldsymbol{N}_{sR} \\ \boldsymbol{N}_{1s} & \boldsymbol{N}_{11} & \boldsymbol{0} & \cdots & \boldsymbol{0} \\ \boldsymbol{N}_{2s} & \boldsymbol{0} & \boldsymbol{N}_{22} & \cdots & \boldsymbol{0} \\ \vdots & & & \ddots & \\ \boldsymbol{N}_{Rs} & \boldsymbol{0} & \boldsymbol{0} & \cdots & \boldsymbol{N}_{RR} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_s \\ \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \vdots \\ \boldsymbol{x}_R \end{bmatrix} = \begin{bmatrix} \boldsymbol{u}_s \\ \boldsymbol{u}_1 \\ \boldsymbol{u}_2 \\ \vdots \\ \boldsymbol{u}_R \end{bmatrix}, \tag{3}$$

with the per-station blocks built from that station's pre-formed design rows,

$$\boldsymbol{N}_{ss}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{s,r}, \quad \boldsymbol{N}_{sr}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{r,r}, \quad \boldsymbol{N}_{rr}^{r} = \boldsymbol{A}_{r,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{r,r}, \tag{4}$$

and $\boldsymbol{u}_s^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{y}_r$, $\boldsymbol{u}_r^{r} = \boldsymbol{A}_{r,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{y}_r$. A full (non-diagonal) $\boldsymbol{W}_r$ is supported, so correlated undifferenced measurement covariance (e.g. shared pre-corrections) propagates exactly (`addStationObservations`, `NetworkNormalEquations.hpp`).

Lossless marginalisation. The arrowhead structure is solved by Schur-eliminating each station's private block onto the shared border. Each $\boldsymbol{N}_{rr}^{r}$ is factorised once ($\mathrm{LDL}^{\mathrm{T}}$) when the station is added; the elimination accumulates

$$\tilde{\boldsymbol{N}}_{ss} = \sum_{r\in\mathcal{R}} \left(\boldsymbol{N}_{ss}^{r} - \boldsymbol{N}_{sr}^{r}\,(\boldsymbol{N}_{rr}^{r})^{-1}\,\boldsymbol{N}_{sr}^{r\mathrm{T}} \right) \;+\; \boldsymbol{N}_{ss}^{\mathrm{prior}}, \tag{5}$$

$$\tilde{\boldsymbol{u}}_s = \sum_{r\in\mathcal{R}} \left(\boldsymbol{u}_s^{r} - \boldsymbol{N}_{sr}^{r}\,(\boldsymbol{N}_{rr}^{r})^{-1}\,\boldsymbol{u}_r^{r} \right) \;+\; \boldsymbol{u}_s^{\mathrm{prior}}, \tag{6}$$

the reduced shared system $\tilde{\boldsymbol{N}}_{ss}\,\boldsymbol{x}_s = \tilde{\boldsymbol{u}}_s$ (`NetworkNormalEquations::reduce`). The reduction is a *lossless* marginalisation, not an approximation: solving Equations (5) and (6) for $\boldsymbol{x}_s$ and back-substituting per station,

$$\boldsymbol{x}_r = (\boldsymbol{N}_{rr}^{r})^{-1}\big(\boldsymbol{u}_r^{r} - \boldsymbol{N}_{sr}^{r\mathrm{T}}\,\boldsymbol{x}_s\big), \tag{7}$$

reproduces the dense stacked solve to machine precision. Fill-in is confined to the satellite border and each station's elimination is independent, so the cost scales linearly in the station count and the per-station eliminations parallelise (`reconstructStation`, `NetworkNormalEquations.cpp`). The shared-block priors $\boldsymbol{N}_{ss}^{\mathrm{prior}}$ in Equation (5), namely soft physical priors and any additive exact-gauge contributions, are added pre-reduction via `addSharedInfo` and carry through the Schur complement unchanged.

Remark 3. This arrowhead-with-exact-reduction topology mirrors the NGS M-PAGES `NgsLsq::NormalEquations::reduce` / `reconstructLocals` batch network solver, specialised so that the M-PAGES "global" block is the shared satellite border and the per-epoch swept block becomes the R independent station blocks. The design panel rejected a two-stage covariance-form alternative precisely because the normal-equation Schur form is exact and cheaper here.

1.6 Constellation scope

The producer estimates GPS and Galileo, with BeiDou supported on the same footing; **GLONASS phase biases are excluded**. The reason is structural, not a tuning choice: GLONASS is FDMA, so each satellite transmits on a slightly different frequency and the receiver inter-channel (hardware) bias does *not* cancel cleanly across satellites the way it does for the CDMA constellations. The satellite/receiver phase-bias bookkeeping that underpins the explicit-bias S-system (Sections 1.3 and 1.4) assumes a per-signal bias common across a constellation's satellites; the per-frequency GLONASS inter-channel bias breaks that assumption and would corrupt the phase datum. GLONASS is therefore excluded from the network *phase*-bias product by design (`docs/ssystem_conformance_2026-06-15.md`); the user side excludes it from ambiguity resolution for the same reason. A *code-only* GLONASS mode is nonetheless supported behind `network.estimate_glonass` (`enableGlonassNetwork`, default off): it admits GLONASS to the solve carrying a satellite clock and a per-slot receiver inter-frequency bias but *no* FDMA phase bias, so the phase-datum bookkeeping above is untouched. The mode is mutually exclusive with the W6 satellite code-bias slice, because GLONASS admission suppresses the GPS/Galileo receiver DCB that code-bias separation needs, so `network.estimate_sat_code_bias` together with `network.estimate_glonass` is a hard load error, and therefore also with the co-estimated SH-VTEC ionosphere, which forces the code-bias slice on. (This is distinct from the float-only GLONASS support the user engine carries for positioning; with the code-only mode off, the network *producer* generates no GLONASS bias products at all.)

1.7 The closed loop: producer → held-out user

The emitted product is one `NetworkSolutionEpoch` per content epoch: per-satellite clock C_0 plus per-signal code and phase biases, keyed by RINEX satellite id and serialised in IGS-SSR convention by `NetworkProductEmitter` (`NetworkEstimator::toSolutionEpoch`). That product is exactly the SSR stream the single-receiver engine already knows how to consume: the satellite clock refines $-c\,\mathrm{d}t^i$ and the observable-specific biases refine $-B^i_s$ and $-b^i_s$ as *known* corrections, restoring the integer ambiguity property exactly as described in the SSR-corrections section.

The validation strategy follows directly from this symmetry. The companion `ProcessingMode::PPP_RTK_User` mode runs the ordinary uncombined PPP-RTK pipeline but consumes the network's own bias/atmosphere product. Holding one station *out* of the network solve and then positioning it as a user against the produced corrections turns the consumer into a closed-loop product test: the held-out station's recovered position and fixed ambiguities certify that the producer's S-system datum and the consumer's integer recovery are mutually consistent. The `PPP_RTK_User` consumer flows through the standard PPP path (it is not a separate estimator); only the producer takes the joint multi-station pipeline of this section.

That test exists in two mechanised forms. The *in-process* harness (`network.closed_loop.enable`) builds a consumer engine from a derived configuration during setup and hands it an *owned* SSR correction store rather than the process-wide one, so the producer's own products can never leak into a store some other component reads; the consumer is then stepped from the emit seam `networkEmitStore`, after that epoch's products have been pushed, which is what makes the loop closed *within* the epoch. Its consumer runs a between-satellite single-difference user model by default, or an undifferenced one under `closed_loop.consumer_undifferenced`. The *offline replay* driver (`network.closed_loop.replay_products_path` → `Application::runConsumerReplay`, `app/ApplicationConsumerReplay`) short-circuits `runNetwork` before any producer setup and fills the same owned store from an already-written product file through the same reader path, so the

recorded stream's discontinuity bookkeeping is reconstructed exactly and a position-domain arm costs minutes rather than a producer run. One boundary is worth stating because it limits what a replay arm proves: the in-process SH-VTEC slant-ionosphere supply has no product-wire record, so `closed_loop.consume_network_iono` is inert on the replay path and the consumer floats its own ionosphere there.

References

[1] D. Odijk. "Fast precise GPS positioning in the presence of ionospheric delays". In: *Publications on Geodesy* 52 (2003).

[2] P. J. G. Teunissen. "Generalized inverses, adjustment, the datum problem and S-transformations". In: *Optimization and Design of Geodetic Networks* (1985), pp. 11–55.